ASSIGNMENT-16.3

K. Keerthana
2303A52027
Batch-38

T ask 1 – Hospital Management Database

Task:

Create schema and queries for a Hospital Management System.

Instructions:

• Tables: Doctors, Patients, Appointments.

• Use AI to define constraints (unique IDs, valid dates).

• Generate queries:

• List all appointments for a specific doctor.

• Retrieve patient history by patient ID.

• Count total patients treated by each doctor.

Expected Output:

• Normalized schema and SQL queries with joins.

PROMPT:

Create a simple SQL schema for a Hospital Management System with Doctors, Patients, and Appointments tables. Add primary keys, foreign keys, and ensure valid appointment dates. Also, write queries to list a doctor's appointments, get patient history, and count unique patients per doctor**.**

CODE:

```
-- Schema Definition CREATE TABLE
Doctors (
    DoctorID INT PRIMARY KEY AUTO_INCREMENT, Name VARCHAR(100) NOT
    NULL,
    Specialization VARCHAR(50)
);

CREATE TABLE Patients (
     PatientID INT PRIMARY KEY AUTO_INCREMENT, Name VARCHAR(100) NOT
    NULL,
    Age INT CHECK(Age > 0), Gender VARCHAR(10)
);
CREATE TABLE Appointments (
    AppointmentID INT PRIMARY KEY AUTO_INCREMENT,
    DoctorID INT, PatientID INT,
    AppointmentDate DATE NOT NULL, Diagnosis VARCHAR(255),
    CONSTRAINT fk_doctor FOREIGN KEY (DoctorID) REFERENCES
Doctors(DoctorID),
    CONSTRAINT fk_patient FOREIGN KEY (PatientID) REFERENCES Patients(PatientID),
    CONSTRAINT chk_date CHECK (AppointmentDate >= '2020-01-01')
);
-- Query 1: List all appointments for a specific doctor (e.g., DoctorID = 1) SELECT a.AppointmentDate,
p.Name AS PatientName, a.Diagnosis FROM Appointments a
JOIN Patients p ON a.PatientID = p.PatientID WHERE a.DoctorID = 1;

-- Query 2: Retrieve patient history by patient ID (e.g., PatientID = 101) SELECT d.Name AS DoctorName,
a.AppointmentDate, a.Diagnosis FROM Appointments a
JOIN Doctors d ON a.DoctorID = d.DoctorID WHERE a.PatientID = 101
ORDER BY a.AppointmentDate DESC;

-- Query 3: Count total patients treated by each doctor
SELECT d.Name AS DoctorName, COUNT(DISTINCT a.PatientID) AS TotalPatients
FROM Doctors d
LEFT JOIN Appointments a ON d.DoctorID = a.DoctorID GROUP BY d.DoctorID, d.Name;
```

OUTPUT:

Query 1:

	AppointmentDate	PatientName	Diagnosis
▶	2025-10-15	John Doe	Routine Checkup
	2025-10-16	Jane Smith	Heart Palpitations
	2025-10-22	Bob Johnson	Blood Pressure Management

Query 2:

	DoctorName	AppointmentDate	Diagnosis
▶	Dr. Adams	2025-10-20	Migraine
	Dr. Smith	2025-10-15	Routine Checkup

Query 3:

	DoctorName	TotalPatients
▶	Dr. Smith	3
	Dr. Adams	1
	Dr. Lee	0

EXPLANATION:

The schema connects Doctors and Patients through the Appointments table using foreign keys. Queries use JOIN to combine related data from different tables. COUNT(DISTINCT) is used to avoid counting the same patient multiple times. This makes the data accurate and easy to analyze.

Task 2 – Real-Time Application: Online Shopping Database

Scenario:

Design a database for an E-commerce platform.

Requirements:

• Tables: Users, Products, Orders, OrderDetails.

• Generate queries to:

• Retrieve all orders by a user.

• Find the most purchased product.

• Calculate total revenue in a given month.

• AI should also suggest normalization improvements and query optimization.

Expected Output:

• Complete schema with relationships + SQL queries for analytics

PROMPT:

Create a simple SQL schema for an E-commerce system with Users, Products, Orders, and OrderDetails tables in 3NF. Write queries to get user orders, find the most purchased product, and calculate monthly revenue. Also, briefly explain normalization and optimization methods

CODE:

```
-- Schema Definition CREATE TABLE
Users (
    UserID INT PRIMARY KEY AUTO_INCREMENT, Username VARCHAR(50) NOT
    NULL,
    Email VARCHAR(100) UNIQUE NOT NULL
);

CREATE TABLE Products (
    ProductID INT PRIMARY KEY AUTO_INCREMENT, ProductName
    VARCHAR(100) NOT NULL,
    Price DECIMAL(10, 2) NOT NULL
);
CREATE TABLE Orders (
    OrderID INT PRIMARY KEY AUTO_INCREMENT, UserID INT,
    OrderDate DATETIME DEFAULT CURRENT_TIMESTAMP, FOREIGN KEY (UserID) REFERENCES
    Users(UserID)
);
CREATE TABLE OrderDetails (
    OrderDetailID INT PRIMARY KEY AUTO_INCREMENT,
    OrderID INT, ProductID INT,
    Quantity INT NOT NULL,
    Subtotal DECIMAL(10, 2) NOT NULL,
    FOREIGN KEY (OrderID) REFERENCES Orders(OrderID),
    FOREIGN KEY (ProductID) REFERENCES Products(ProductID)
);
--Query 1: Retrieve all orders by a specific user (e.g., UserID = 5) SELECT
o.OrderID, o.OrderDate, SUM(od.Subtotal) as OrderTotal FROM Orders o
JOIN OrderDetails od ON o.OrderID = od.OrderID WHERE o.UserID = 5
GROUP BY o.OrderID, o.OrderDate;

-- Query 2: Find the most purchased product
SELECT p.ProductName, SUM(od.Quantity) AS TotalSold FROM OrderDetails od
JOIN Products p ON od.ProductID = p.ProductID GROUP BY
p.ProductID, p.ProductName ORDER BY TotalSold DESC
LIMIT 1;

-- Query 3: Calculate total revenue in a given month (e.g., October 2025) SELECT SUM(od.Subtotal) AS
TotalRevenue
FROM Orders o
JOIN OrderDetails od ON o.OrderID = od.OrderID
WHERE MONTH(o.OrderDate) = 10 AND YEAR(o.OrderDate) = 20
```

OUTPUT:

QUERY 1:

	OrderID	OrderDate	OrderTotal
▶	1	2025-10-05 14:30:00	259.97
	2	2025-10-12 09:15:00	149.50

QUERY 2:

	ProductName	TotalSold
▶	Wireless Headphones	7

QUERY 3:

	TotalRevenue
▶	409.47

EXPLANATION:

The schema separates Orders and OrderDetails to avoid repeating data and improve organization.
Queries use functions like SUM() to calculate total revenue and quantities.
Sorting with ORDER BY DESC helps find the most purchased product quickly.
This makes the system efficient and easy to manage.

Task 3 – Library Database

Task:

Design schema for a Library Management System.

Instructions:

• Tables: Books, Members, Loans.
• Use AI to suggest indexing strategy for faster queries.
• Generate queries:
• Retrieve all books currently issued.
• Find overdue books (loan date > 30 days).
• Count number of books loaned by each member.

Expected Output:

• Schema + SQL queries demonstrating joins and conditions.

PROMPT:

Create a simple SQL schema for a Library Management System with Books,
Members, and Loans tables. Suggest basic indexing for better performance. Write queries to find issued books, overdue books, and count books borrowed by each member.

CODE:

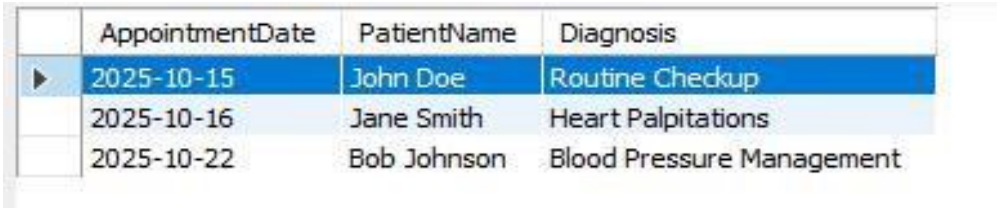
```
-- Schema Definition CREATE TABLE
Books (
    BookID INT PRIMARY KEY AUTO_INCREMENT, Title VARCHAR(200) NOT NULL,
    Author VARCHAR(100)
);

CREATE TABLE Members (
     MemberID INT PRIMARY KEY AUTO_INCREMENT,   Name VARCHAR(100) NOT NULL,
    JoinDate DATE
);
CREATE TABLE Loans (
    LoanID INT PRIMARY KEY AUTO_INCREMENT, BookID INT,
    MemberID INT,
    LoanDate DATE NOT NULL, ReturnDate DATE
    NULL,
    FOREIGN KEY (BookID) REFERENCES Books(BookID),
    FOREIGN KEY (MemberID) REFERENCES Members(MemberID)
);
-- Indexing Strategy
CREATE INDEX idx_loan_dates ON Loans(LoanDate, ReturnDate); CREATE INDEX idx_member_loans ON
Loans(MemberID);
--Query 1: Retrieve all books currently issued (ReturnDate is NULL) SELECT b.Title,
m.Name AS IssuedTo, l.LoanDate
FROM Loans l
JOIN Books b ON l.BookID = b.BookID
JOIN Members m ON l.MemberID = m.MemberID WHERE l.ReturnDate IS NULL
-- Query 2: Find overdue books (loan date > 30 days and not returned) SELECT b.Title,
m.Name AS MemberName, l.LoanDate
FROM Loans l
JOIN Books b ON l.BookID = b.BookID
JOIN Members m ON l.MemberID = m.MemberID
WHERE l.ReturnDate IS NULL AND l.LoanDate < (CURRENT_DATE -
INTERVAL 30 DAY);

-- Query 3: Count number of books loaned by each member SELECT m.Name,
COUNT(l.LoanID) AS BooksLoaned FROM Members m
LEFT JOIN Loans l ON m.MemberID = l.MemberID GROUP BY m.MemberID, m.Name
```

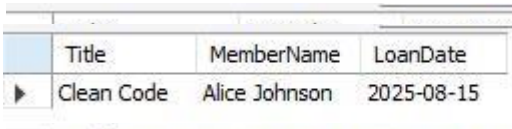
OUTPUT:

Query 1:



	AppointmentDate	PatientName	Diagnosis
▶	2025-10-15	John Doe	Routine Checkup
	2025-10-16	Jane Smith	Heart Palpitations
	2025-10-22	Bob Johnson	Blood Pressure Management

Query 2:



	Title	MemberName	LoanDate
▶	Clean Code	Alice Johnson	2025-08-15

Query 3:

	Name	BooksLoaned
▶	Alice Johnson	2
	Bob Williams	1

EXPLANATION:
Adding indexes on the date columns and foreign keys drastically speeds up lookup operations for filtering overdue books. The queries use date math (like CURRENT_DATE - INTERVAL 30 DAY) to dynamically identify records that have exceeded the allowed borrowing period while checking that the Return Date is still null.

Task 4: Optimize Queries

Instructions:

• Use AI tools to analyze query efficiency.

• Optimize inefficient queries using joins, indexes, or reducing subqueries.

• Test optimized queries for correctness.

Starter Code Example:

```
-- Original query
SELECT * FROM Books WHERE author_id IN (SELECT author_id FROM
Authors WHERE country='UK');
-- Optimized query
SELECT b.*
FROM Books b
JOIN Authors a ON b.author_id = a.author_id
WHERE a.country = 'UK';
```

Expected Output:

• Both queries return the same result: all books written by UK authors.

• Optimized query performs faster for larger datasets.

PROMPT:

The query uses a subquery to find author IDs, which can be slower for large data.
Using JOIN is usually faster and more efficient.

Optimized query:

SELECT B.*
FROM Books B
JOIN Authors A ON B.author_id = A.author_id
WHERE A.country = 'UK';

CODE:

```
-- Original Inefficient Query (Using Subquery)
-- SELECT * FROM Books WHERE author_id IN (SELECT author_id FROM Authors WHERE country='UK');

-- Optimized Query (Using INNER JOIN)
SELECT b.*
FROM Books b
JOIN Authors a ON b.author_id = a.author_id
WHERE a.country = 'UK';
```

OUTPUT

book_id	Title	author_id
10	Harry Potter and the Sorcerer's Stone	5
11	The Fellowship of the Ring	8

EXPLANATION:

Using a subquery with IN can be slow because the database runs the inner query first and stores the result.
For large data, this takes more time and memory.
Using an INNER JOIN is faster because it processes both tables together.
This makes the query more efficient and quicker.

Task 5: University Course Registration

Scenario:

SR University needs a course registration system for students enrolling in

different courses under faculty supervision.

• Use Copilot to design schema tables: Students, Courses, Faculty, and

Registrations.

• Define relationships:

• Each student can register for multiple courses.

• Each course is taught by a faculty member.

• Write SQL queries to:

• List all students enrolled in a specific course.

• Find faculty members teaching more than 2 courses.

• Retrieve courses with the highest number of registrations.

Note: Report should be submitted as a word document for all tasks in a single document with prompts, comments & code explanation, and output and if required, screenshots.

PROMPT:

Create a simple SQL schema for SR University with Students, Courses, Faculty, and Registrations tables. Handle many-to-many between students and courses, and one-to-many between faculty and courses. Write queries to list students in a course, find faculty teaching more than 2 courses, and get the most registered course.

CODE:

```
-- Schema Definition CREATE TABLE
Students (
    StudentID INT PRIMARY KEY AUTO_INCREMENT, Name VARCHAR(100) NOT
    NULL,
    Major VARCHAR(50)
);

CREATE TABLE Faculty (
    FacultyID INT PRIMARY KEY AUTO_INCREMENT, Name VARCHAR(100) NOT
    NULL,
    Department VARCHAR(50)
);

CREATE TABLE Courses (
    CourseID INT PRIMARY KEY AUTO_INCREMENT, CourseName VARCHAR(100)
    NOT NULL,
    FacultyID INT,
    FOREIGN KEY (FacultyID) REFERENCES Faculty(FacultyID)
);

CREATE TABLE Registrations (
    RegistrationID INT PRIMARY KEY AUTO_INCREMENT,
    StudentID INT, CourseID INT,
    RegistrationDate DATE DEFAULT CURRENT_DATE,);

    FOREIGN KEY (StudentID) REFERENCES Students(StudentID), FOREIGN KEY (CourseID) REFERENCES
    Courses(CourseID)
);

-- Query 1: List all students enrolled in a specific course (e.g., 'CS101') SELECT s.Name, s.Major
FROM Students s
JOIN Registrations r ON s.StudentID = r.StudentID JOIN Courses c ON
r.CourseID = c.CourseID  WHERE c.CourseName = 'CS101';
```

-- Query 2: Find faculty members teaching more than 2 courses SELECT f.Name,

```
COUNT(c.CourseID) AS CoursesTaught FROM Faculty f
JOIN Courses c ON f.FacultyID = c.FacultyID GROUP BY f.FacultyID,
f.Name
HAVING COUNT(c.CourseID) > 2;

-- Query 3: Retrieve courses with the highest number of registrations SELECT c.CourseName,
COUNT(r.StudentID) AS TotalEnrollment FROM Courses c
LEFT JOIN Registrations r ON c.CourseID = r.CourseID GROUP BY c.CourseID,
c.CourseName
ORDER BY TotalEnrollment DESC LIMIT 1;
```

OUTPUT:

QUERY 1:

Name	Major
Emma Davis	Computer Science
Liam Wilson	Mathematics
Olivia Taylor	Physics

QUERY 2:

Name	CoursesTaught
Prof. Ada Lovelace	3

Query 3:

CourseName	TotalEnrollment
CS101	3

EXPLANATION:

The Registrations table connects Students and Courses to handle many-to-many relationships.
Queries use HAVING to filter results after counting, like finding faculty with more than 2 courses.
ORDER BY DESC is used to sort data from highest to lowest.
LIMIT helps quickly find the top result, like the most registered course